

Proposal “Road to GM”

[RAQUEL M QUINONES-VELEZ](#)

[JASEY J ACEVEDO-SALDANA](#)

1) Introduction

Chess is a game of logic that requires players to think beyond their own moves and anticipate their opponent’s plans. As a result, decisions are made based on analysis and strategy rather than impulse. [1].

Due to these characteristics, chess is widely used as an educational tool to promote critical thinking, logical reasoning, and problem-solving skills [1]. A SciELO research paper about neuroscience and chess has shown that structured chess instruction can positively influence student’s concentration, planning skills, memory skills and more [2]. As a result, the game of chess is commonly incorporated into school programs, clubs, and self learning platforms.

In recent years, particularly in December 2023, platforms such as Chess.com experienced a significant increase in users, with player numbers reportedly doubling [3]. This growth has led to an increased demand for educational, software-based tools that provide structured lessons and interactive exercises.

In Puerto Rico, chess maintains an active community. According to the International Chess Federation (FIDE), there are approximately 1,637 registered players affiliated with the Puerto Rico federation [4]. Something to note is that this number reflects only officially rated competitors and does not account for recreational players, school participants, and independent learners who are not formally registered. Therefore, the true number of active chess players in Puerto Rico is likely significantly higher.

One tool that previously supported structured chess education in Puerto Rico was Chessimo [5], an application that offered instructional modules to help users improve their chess skills through one or two step exercises. Despite its educational value, the application has become outdated and has been removed from major platforms such as Google Play, and its desktop version is no longer available. As a result, learners and educators who relied on this platform no longer have access to its content or learning approach.

The goal of this project is to recreate and improve upon the original application’s design in order to enhance users’ chess knowledge through bite-sized exercises derived from larger chess concepts. This goal will be achieved by developing a web-based platform using technologies such as React with react-chessboard for the user interface and chess.js for game logic and move validation. The backend will be built using Node.js and Express.js, with database support through MongoDB and/or SQL. Development tools such as NPM, nodemon, and package.json dependencies will be used for project management, and Docker may be utilized for deployment and infrastructure. All of this to provide a more accessible, interactive, and engaging learning experience.

In addition to replicating the core learning functionality of the original application, the new system will focus on a key new feature: modular user-created instructional content. The platform will allow teachers, students, and independent learners to create, organize, and share their own chess modules and exercises. Alongside this central feature, the system will also introduce modular lesson management and improved interactive chessboards utilizing modern technologies. Together, these enhancements aim to deliver a more flexible, scalable, effective, and creative learning environment for teaching chess across different skill levels.

2) Problem Statement:

Chessimo was a valuable learning resource for beginning and intermediate chess players in Puerto Rico. It contributed to the development of multiple strong players, including Eliam Gerena Rivas, who became Puerto Rico Champion in 2022 and undefeated youth champion that same year, as stated by the former President of FAPR [6]. However, after Chessimo was removed from Google Play and its PC version was discontinued, this structured teaching resource became inaccessible.

Not only has it been removed, Chessimo also presented several technical limitations. The platform lacked cross-device synchronization, meaning user progress and completed exercises were not stored across devices. Additionally, users were unable to combine modules (e.g., opening, middlegame, and endgame) into training sessions, limiting customization and flexibility in structured learning. The platform also provided limited options for board and piece customization and did not support dynamic data persistence for long-term user progress tracking [6].

This project does not aim to create a carbon copy of Chessimo but rather to recreate its core teaching functionality while addressing some of these technical limitations and expanding its capabilities. The proposed system will implement persistent user accounts, customizable training modules, and user modular content.

This project will primarily benefit chess clubs, schools, and competitive players seeking structured training tools. Potential stakeholders include the Federación de Ajedrez de Puerto Rico (FAPR) [7], public and private schools with chess teams, local tournaments, and independent chess clubs. Casual players may benefit from the platform's foundational training features, while advanced users and communities can leverage collaborative modules to maximize the platform's potential.

3) Project Objectives:

Objective	Specific	Measurable	Achievable	Relevant
User-Created Modular Instructional Content	Enable users to create, edit, and publish chess modules and lessons	Up to 4 modules per user , with ≥5 exercises per module	<u>CRUD</u> [8] system with authentication and role-based access control	Core differentiating feature enabling customizable instruction
Interactive Chess Exercise System	Provide playable chess exercises with move validation and progress tracking	90% of attempted exercises tracked and ≥90% legal move validation	Chess logic implemented using chess.js	Forms the functional backbone of the learning platform

Default Structured Learning Modules	Provide a predefined set of chess learning modules	At least 4 default modules (openings, tactics, strategy, endgames)	Curated exercises stored and delivered through the platform	Ensures immediate usability for new users
Role-Based Access and Group/Class System	Support teacher and student roles with group-based learning	Users can create and join at least one class or group	Authentication and permissions enforced at the API level	Enables real world use in schools and chess clubs

Table 1: Road to GM SMART Objectives

The objective for User-Created Modular Instructional Content testing will verify that users can successfully create, edit, update, and delete modules through a fully functional CRUD system with authentication and role-based access control. This will be measured by us by replicating what a user could do, creating up to four modules, each containing exercises and that it can be stored. For the Interactive Chess Exercise System objective, at least 90% of user move attempts are correctly tracked and recorded in the database, and that legal move validation behaves consistently with the rules enforced by chess.js. Testing will involve simulating user gameplay scenarios and comparing expected outcomes against actual system responses. The objective for Default Structure Learning Modules will be validated by confirming the presence of at least four predefined modules accessible upon account creation. Finally, the Role-Base Access and Group/Class system objective will be tested by verifying that users can successfully create and join classes, and that access permissions are enforced at the API level, preventing unauthorized data access.

4) Solution Approach:

The project will develop a chess training platform that enables users to create and share structured exercise modules. While platforms such as Chess.com [9] provide already made tactical exercises, our system will focus on allowing users to organize and distribute their own training sets. The sharing functionality enables instructors, clubs, and advanced players to distribute curated modules to others, supporting structured and collaborative learning. An expected drawback to this approach would be the effort required to create a routine that accomplishes this. However, if the platform has sharing capabilities, a struggling player could reach out and receive relevant training routines made by more adept players or even a personal tutor.

The value proposition of this solution is that it focuses on personalized and reusable chess training. It allows users to create, organize, and share custom exercise sets. The platform will enable targeted improvement based on each individual weakness rather than relying solely on random exercises. This makes the system especially valuable as a long term training and instructional tool rather than a one time practice resource. Unlike the original Chessimo [5] platform, the proposed system will implement persistent user accounts and data storage, ensuring that exercises, progress, and completed modules are synchronized across devices. Additionally, the system will allow a flexible combination of training modules and personalized routines, addressing previous structural limitations.

Feature	Chessimo [5]	Chess.com [9]	Lichess [10]	Road to GM
Predefined Tactical Exercises	Yes	Yes	Yes	Yes
User-Created Custom Modules	No	No (limited study features)	Limited (Studies, not structured training modules)	Yes (core feature)
Combination of Modules and Personalized Routines	No	No	No	Yes
Cross-Device Synchronization	Limited / No	Yes	Yes	Limited/ Future Long-Term Feature Goal
Progress Tracking	Basic	Yes	Yes	Yes (modular + class-based tracking)
Class/Group Management	No	Limited (premium tools)	No	Yes (teacher student roles)

Table 2: Compassion Table of Chess Training Platforms

Additionally, the effectiveness of shared content depends on the quality of exercises created by the users. These limitations are mitigated by offering basic content for beginners to start their journey and by sharing routines created by experienced players or instructors, with the support of the FIDE and other clubs around the island.

The resources required to bring the solution into operation include access to modern web development tools and infrastructures. This includes a backend web server environment built on Node.js with the Express framework to handle business logic and API communication. A relational

database system such as PostgreSQL (or alternatively MongoDB) will be required to securely store user accounts, exercises, modules, and progress data. The system will also require hosting infrastructure, such as a virtual private server or platform service provider, to ensure accessibility, scalability, and secure HTTPS communication. It also requires a development team responsible for frontend and backend integration, version control systems such as Git (following a structured branching strategy), and testing frameworks for validating API endpoints, database integrity, and frontend functionality. For long-term operation, the system will require ongoing server hosting, periodic security updates, database maintenance, and monitoring to ensure performance and data reliability. These resources ensure that the proposed platform is technically feasible, maintainable, and scalable within the scope of the project.

The intellectual property considerations is that this project makes use of open sourced libraries such as chess.js and react-chessboard, with open source licenses. All original source code, system design and platform specific features developed as part of this project will constitute as newly created intellectual property. User created exercises and instructional modules remain the intellectual property of their creators. The platform serves as a hosting and distribution intermediary while maintaining ownership attribution for user-generated instructional content.

5) Technical Approach:

Our project will be implemented in a 3-tiered layered architecture pattern. By the nature of the domain we are working with, it divides into these distinct concerns: Presentation, Business Logic and Data Persistence, for more information refer to Figure 1. This architecture approach allows for separate layers to be tested without affecting each other which helps with maintainability, meaning it promotes modularity and testability by ensuring that each layer has a clearly defined responsibility and can communicate only with the adjacent layers.

This architecture aligns with the multi-tier web application design principles [11].

Layer	Responsability	Technology	Justification
Presentation (UI)	Handles user interaction, renders the chess board and game state	React with react-chessboard	React enables component based UI development and react-chessboard has prebuilt chess UI components and handling of user interaction. Reducing development time and complexity.
Business Logic	Validates game state, enforces chess rules, handles exercise logic	Node.js with Express, and chess.js	Chess.js is a well tested open source chess engine that handles game state, rule/move validation. Compatible with the UI technologies mentioned before.

Data Layer	Storing users accounts, exercises, modules and progress	PostgresSQL	Reliable when storing large amounts of text data, support for structured relationships between users, modules, and exercises. As well it integrates easily with Express.
-------------------	---	-------------	--

Table 3:

Multi-tier Web Application Architecture for Road to GM.

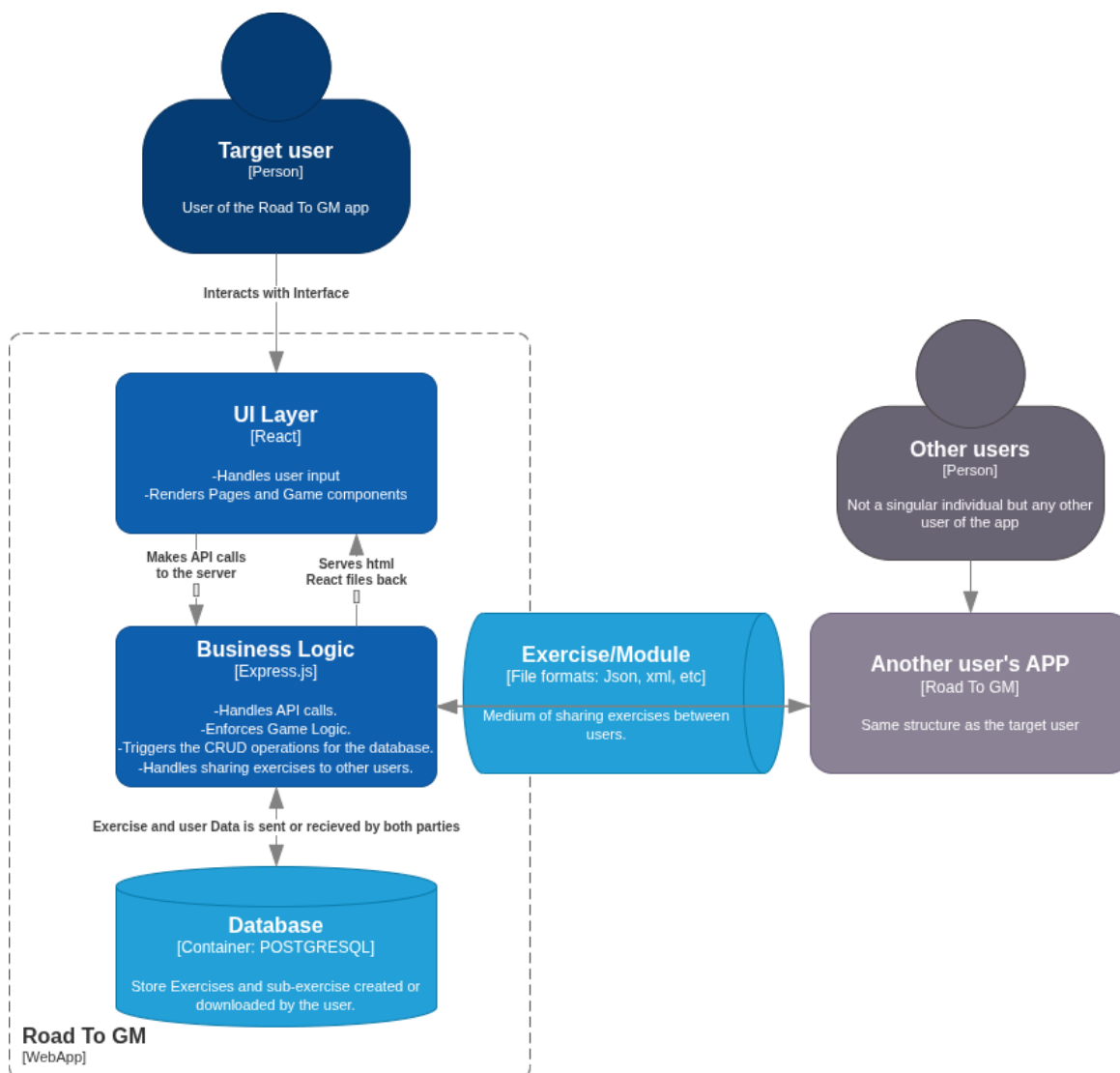


Figure 1: Multi-tier Web Application Diagram

Each user's architecture follows the same 3 layers pattern (React, Express, PostgreSQL). Communication between layers will only happen through adjacent layers. The UI layer will communicate with business logic through RESTful API endpoints and the business layer will

communicate with the database with queries. At no point will the UI be communicating directly with the database, this keeps everything modular and easy to maintain. Communication between users is handled by the users' business logic layers; some file formats, like JSON, will be used for this task because of their efficiency with data transfer.

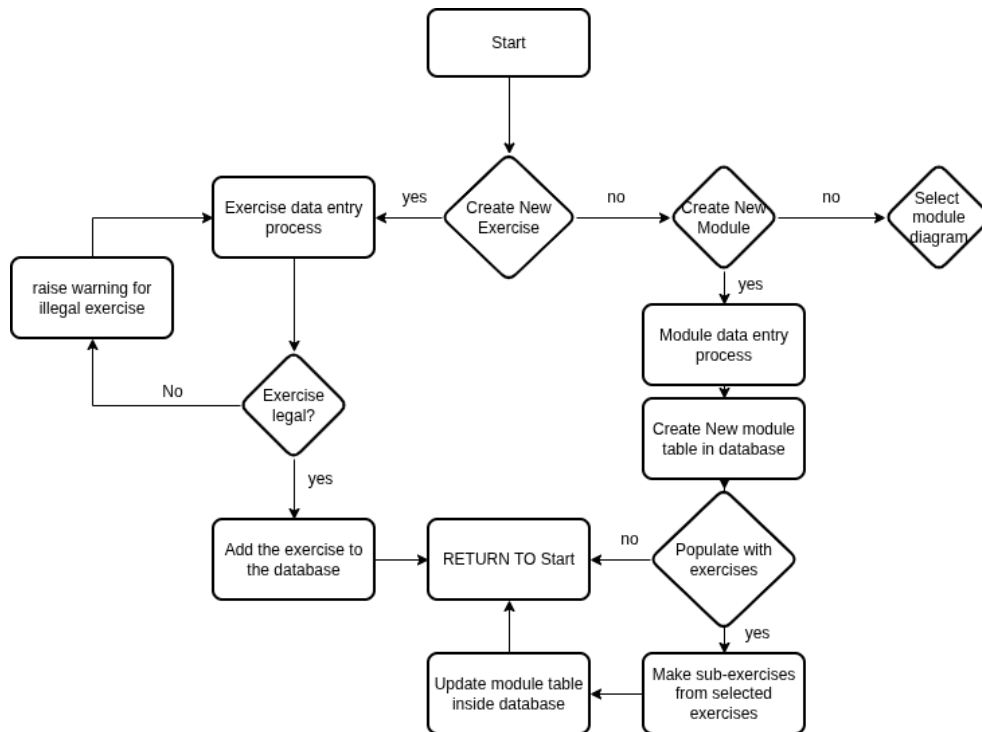


Figure 2: Business Logic FlowChart (First part)

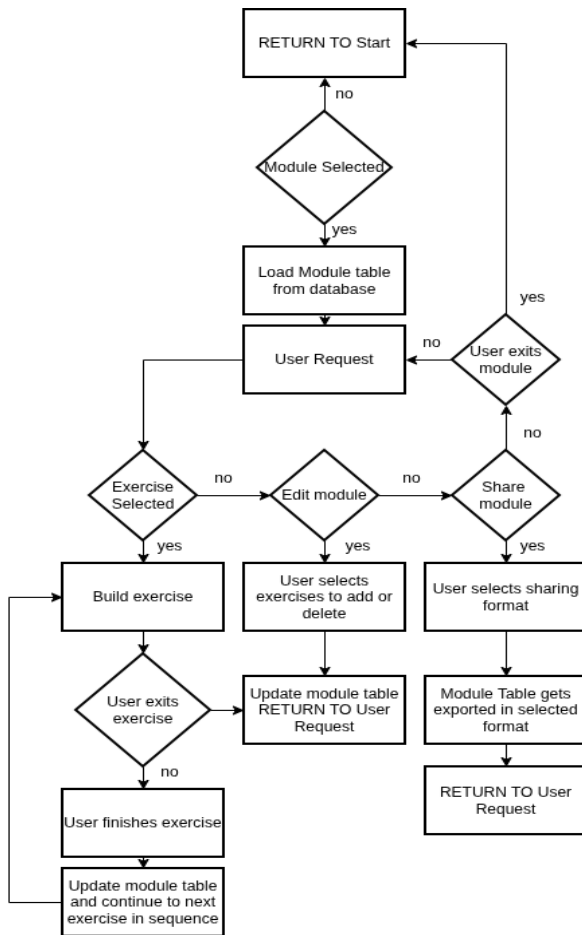


Figure 3: Business logic Module selected (Second Part)

The general workflow of the business logic starts with the users deciding between a few options: do they want to create an exercise (Other CRUD operations omitted from Figure 2 for simplicity sake), create a Module or select a Module. Creating either a module or exercise will trigger the appropriate data entry so they can add a valid exercise or module to their database correctly. If a module is selected, reference Figure 3, the user will be able to edit its contents, share it or play one of its exercises. After the user finishes this operation, any meaningful changes will be updated in the database.

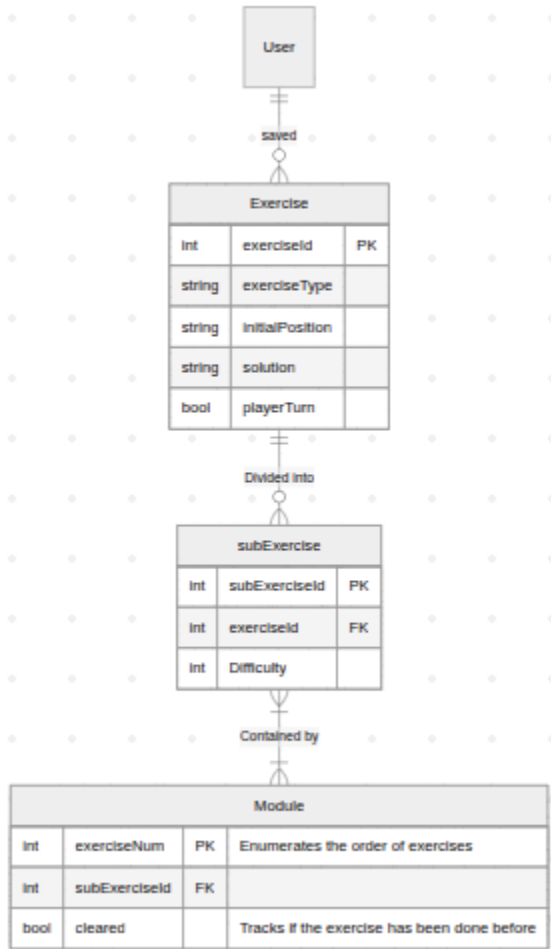


Figure 4: Representation of exercise structure inside Database

Any exercise, no matter the type, can be represented by two crucial characteristics: Initial position and Solution. Chess positions are represented using FEN (Forsyth-Edwards Notation), and solutions, using PGN (Portable Game Notation); both are standardized chess formats [12], [13]. The use of FEN allows the system to store board states as compact strings, enabling efficient data handling and transmission between layers. Furthermore, research has demonstrated that FEN strings can be algorithmically updated without requiring full board-array representation, improving computational efficiency in chess-based systems [14]. The exercise data will be stored in the database using the structure described in Figure 4; exercises will be separated into sub-exercises that can be played inside the modules. All important information will be held by the original exercises avoiding storing the same data multiple times and making the sub-exercise creation faster and simpler.

a) Related Work:

As to related works there's several existing platforms that provide structured chess training. Chessimo [5] offered modular instructional exercises focused primarily on short tactical sequences. However the architecture did not support persistent cross-device synchronization, dynamic module customization, or user-generated structured modular classes.

On the other hand, modern platforms such as Chess.com [9] and Lichess [10] provide extensive tactical training databases and interactive chessboards. These systems typically deliver exercises from large curated problem pools using randomized or rating-based selection algorithms [15], [16]. While effective for tactical exposure, their training structures are primarily system-curated

and do not emphasize user-authored, reusable, and composable instructional modules. Although Lichess supports “Studies” and Chess.com provides puzzle collections, these features are not designed as modular pedagogical pipelines that can be systematically combined into progressive training routines [16].

From a technical standpoint, most existing platforms prioritize content delivery and matchmaking infrastructure over customizable instructional architecture. They do not expose mechanisms for structured user-defined curriculum assembly or persistent module composition. The proposed system differentiates itself by introducing a modular training architecture that allows users to define, store, combine, and share structured exercise pipelines, supported by persistent account-based synchronization and flexible module composition.

b) Engineering Constraints

This project will build upon established chess data standards (FEN/PGN) [12], [13] and will leverage open-source libraries such as chess.js [17] and react-chessboard [18] to recreate and expand on these concepts. It will also follow security practices aligned with the OWASP web development security guidelines [19]. In engineering design, constraints define the practical limitations that shape system architecture, scope, and implementation trade-offs [20].

The development of Road to GM is subject to both technical and organizational constraints. The most significant one is the limited development timeline, as the project must be fully designed, implemented, tested, and documented within the 2026 Spring semester. This limitation will require a strict scope control, and prioritization of the main features. Features that are not critical to the core functionality must be put on hold to ensure the successful completion within the available timeframe. The non core features will be in development after the semester’s completion and in the far future.

Additionally, the Road to GM team consists of two developers rather than three, which limits parallel development capacity. This constraint influences a simpler architecture, and feature prioritization to ensure feasibility within the team’s capacity.

The system constraints are therefore divided into functional (must haves) and secondary (could-have) requirements.

Functional Constraints are the features that are essential for the project to operate and fulfill its core value proposition:

- Basic predefined set of chess exercises: Required for immediate usability for users upon first access.
- Legal move validation: Required for correctness and rule enforcement; implemented using chess.js [17]
- Exercise/module sharing capabilities: Required to enable reuse of instructional content among clubs, tutors, and students.
- Users-created exercises and modules: Required because this is the core differentiating feature of the system.
- Data persistence between sessions: Required to prevent loss of progress and to enable future cross-device usage.

These functional constraints define the minimum viable product necessary for the system to be considered successful.

Secondary Constraints are the features that improve user experiences and adoption but are not strictly required for initial deployment:

- Intuitive User Interface: Encourages user engagement and reduces learning friction.
- Tutorial explaining the rules of the game: Improves accessibility for new players.
- Multi-platform accessibility: (Web first design, mobile compatible layout) Expands potential user reach while prioritizing web deployment as the primary target.

Given the semester timeline and two-member development team, these secondary constraints may be implemented gradually if core functional requirements are completed successfully.

To reduce development time due to the limited timeline, team size, and constraints; the system will emphasize the use of well-established open-source libraries, as mentioned in other sections. It will also employ incremental sprint based development following the Agile Sprint, which is part of the Scrum Framework, with the use of Agile Principles as seen in Table 4 [20]

c) Engineering Standards

The project will adhere to known software engineering standards to ensure maintainability, security, consistency and long-term sustainability. These standards will be about the system architecture, code structure, documentation practices, security implementation, data integrity and version control.

For the system architecture it will follow a layered (three-tier) pattern. This design, as stated in Figure 1, ensures that presentation, business logic, and data layer remain loosely coupled and independently testable. The REST architectural principles [21] are adopted to standardize communication between the layers using HTTP interactions like GET, POST, PUT, DELETE and status codes like 200, 201, 400, 401, 404, 500 [22]. This will help simplify frontend and backend integration and align with modern web development practices.

For the coding standard and code quality to maintain readability and consistency across the development team, the project will adopt the JavaScript and web development naming conventions. In the backend and frontend codebase, camelCase [23] will be used for variables and functions, while PascalCase [23] will be used for React components and class definitions. API route paths will follow the kebab-case [23] formatting to maintain clarity and consistency in endpoints naming.

For the documentation standard the code will include JSDoc-style [24] comments for major functions, clearly describing parameters, return values, and function behavior. The Readme.md documentation will explain the core functionality of the project, the system architecture, and the technologies used (React, Node.js, Express, PostgreSQL/MongoDB, chess.js, and react-chessboard). It will also provide step-by-step instructions to facilitate installation, environment configuration, database setup, and application execution for future development teams.

For security implementation the project will align with the OWASP top ten guidelines [25]. All critical validations and authorization checks will be performed server side to prevent the tampering from the client side. Authentication will follow secure session management practices, including password hashing, and role based access control (RBAC) [26].

The Database Security will follow proper key relationships between users, modules and exercises; as well as the use of parameterized queries to prevent SQL injections, keeping the data safe. This will improve performance, consistency and data reliability.

For version control standards, the project will follow the GitFlow branching strategy [27]; main branch for stable production of code, develop branches for integration and feature branches for isolated development. This will help minimize integration conflicts, reduce the risk of unstable deployments and keep the overall project organized.

d) Best Practices

This project will adopt several software development and engineering best practices, like the separation of concerns through a layered architecture; ensuring that presentation, business logic, and data layers remain loosely coupled and independently testable. Development is conducted using an iterative Agile Sprint approach, allowing incremental feature delivery and early validation within the semester timeline.

Critical operations such as authentication and move validation are enforced through server side validation to prevent client side manipulation. The project will leverage open source libraries to reduce implementation risk and focus development effort on system-specific innovation. The frontend components will be designed using a modular and reusable structure, reducing duplication and simplifying future extension. The API endpoints will be tested independently to ensure correct request handling and response behavior.

These best practices support the system reliability, and the long-term sustainability beyond the academic term [28].

6) Project Plan:

The project will follow Agile principles using a Scrum-based sprint framework, with work organized into two-week sprints [20], [29]. This approach was selected due to the limited one-semester timeline and the two-member team structure. The sprint based model allows incremental development, continuous evaluation of progress, and early identification of issues. At the end of each sprint, completed features will be reviewed and plans adjusted if necessary; ensuring steady progress toward core functionality while allowing flexibility in scope management.

Milestone	Phase	Sprints	Key Activities	Deliverables	Timeline
Pre Project Proposal	Pre-Planning	Sprint 1	Define problem, project objectives, approach, clients	Pre Proposal Document	Week 1-2 February 9,2026
Project Proposal	Planning	Sprint 2	Define problem, requirements, architecture, technical approach related work	Proposal document	Weeks 4-5 February 19,2026
Implement Web Architecture	System Design	Sprint 3	Architecture Implementation, and diagrams	Raquel: Web Interface	Weeks 6–7
Database Implementation	System Design	Sprint 3	Database design, implementation with dummy data	Jasey: Database	Week 6-7

UI + DB Connection	System Design	Sprint 3-4	UI Research, Implementation, DB connection	Raquel + Jasey: Database Integration	Week 7-8
Chess Logistic and Structure	Game Design	Sprint 4	Chess game logistics, modules	Raquel + Jasey: Structured Modules	Week 8-10
Progress Report	Core Development	Sprint 4 -5	Auth, roles, module viewing, chessboard integration	Midterm report Document	Weeks 10-11 April 16, 2026
Feature Complete	Modular Feature	Sprint 5–6	User-created modules, classes, permissions	Raquel + Jasey: Role-Based Access	Weeks 11–14
Final Report	Testing & Refinement	Sprint 7	Testing, bug fixes, performance tuning	Testing + Final Report Document	Weeks 15–16 May 18,2026
Elevator Pitch	Presentation Prep	Sprint 8	Refine value proposition, slides, demo script	Elevator pitch	Week 16 TBD
Final Presentation	Deployment	Sprint 9	Deployment, final demo, polish	Final presentation & demo	Week 17 TBD

Table 4: Road to GM Project Plan (Jan-May 2026)

The testing strategy will be implemented throughout the entire development lifecycle to ensure that the system satisfies its functional requirements and engineering constraints. Throughout the sprints there will be structured testing of the features developed during that iteration, such as the core components. The testing plan will include integration testing to validate communication between the frontend, backend and the other layers; as well as performance testing to ensure exercises load with the correct logic. Complete user workflow testing will include assessment of all the capabilities required to create a module, complete exercises, store progress , and ensure share accessibility. Any issues identified during testing will be documented.

7) Previous Works and References:

[1] Waleed Naeem (OnlineChessTeacher, “Chess and Critical Thinking: Improving Problem-Solving Skills,” *Chess.com*, Nov. 08, 2024.
<https://www.chess.com/blog/OnlineChessTeacher/chess-and-critical-thinking-improving-problem-solving-skills>

- [2] R. Ortiz-Pulido, R. Ortiz-Pulido, L. I. García-Hernández, C. A. Pérez-Estudillo, and M. L. Ramírez-Ortega, "Neuroscientific evidence support that chess improves academic performance in school," *Revista Mexicana de Neurociencia*, vol. 20, no. 4, Aug. 2019, doi: <https://doi.org/10.24875/rmn.m19000060>.
- [3] C. com Team (CHESScom), "Chess Is Booming! and Our Servers Are Struggling.," *Chess.com*, Jan. 23, 2023. <https://www.chess.com/blog/CHESScom/chess-is-booming-and-our-servers-are-struggling>
- [4] "Puerto Rico - FIDE federation • lichess.org," *Lichess.org*, 2026. https://lichess.org/fide/federation/Puerto_Rico (accessed Feb. 19, 2026).
- [5] J. Weiss, "Chessimo.com," 2020. <http://chessimo.com/>
- [6] M. Garcia, Former President, Federación de Ajedrez de Puerto Rico, interview by Jasey Acevedo & Raquel Quiñones, Feb. 2026. https://drive.google.com/file/d/16tQb1qLbDCQsJRcZj6DeVUahnIPB_AdT/view?usp=drive_link
- [7] "Federación de Ajedrez de Puerto Rico," *FAPR*, 2026. <https://www.lafapr.org/> (accessed Feb. 19, 2026).
- [8] "CRUD," *Wikipedia*, Oct. 22, 2019. <https://es.wikipedia.org/wiki/CRUD>
- [9] chess.com, "Chess.com - Play Chess Online - Free Games," *Chess.com*, 2019. <https://www.chess.com/>
- [10] "The best free, adless Chess server," *lichess.org*, Jun. 27, 2019. <https://lichess.org/>
- [11] IBM, "Three-tier architecture," *Ibm.com*, Oct. 18, 2021. <https://www.ibm.com/think/topics/three-tier-architecture>
- [12] "FEN (Forsyth-Edwards Notation) - Chess Terms." *Chess.com*, www.chess.com/terms/fen-chess.
- [13] "Chess PGN (Portable Game Notation)." *Chess.com*, 2026, www.chess.com/terms/chess-pgn.
- [14] A. Iqbal, "An Algorithm for Automatically Updating a Forsyth-Edwards Notation String Without an Array Board Representation," *arXiv.org*, 2020. <https://arxiv.org/abs/2009.03193> (accessed Feb. 19, 2026).
- [15] P. Pinhata, "The Complete Guide To Chess.com Features," *Chess.com*, Feb. 03, 2022. <https://www.chess.com/article/view/chesscom-features>
- [16] "Lichess features," *lichess.org*, 2026. <https://lichess.org/features>.
- [17] ibrahimdenizz, "GitHub - ibrahimdenizz/chess-js: Javascript chess library for chess rules and chess ai," *GitHub*, 2025. <https://github.com/ibrahimdenizz/chess-js>
- [18] Clariity, "GitHub - Clariity/react-chessboard: A modern, responsive chessboard component for React applications.," *GitHub*, Feb. 10, 2026. <https://github.com/Clariity/react-chessboard>.
- [19] OWASP, "OWASP Application Security Verification Standard," *owasp.org*, 2024. <https://owasp.org/www-project-application-security-verification-standard/>
- [20] N. Beecroft, "Agile Sprints: All You Need To Know," *Qa.com*, Feb. 2023. <https://www.qa.com/resources/blog/agile-sprints/>
- [21] "Fielding Dissertation: CHAPTER 5: Representational State Transfer (REST)," *Gbiv.com*, 2025. https://roy.gbiv.com/pubs/dissertation/rest_arch_style.htm
- [22] Mozilla, "HTTP response status codes - HTTP | MDN," *MDN Web Docs*, Mar. 13, 2025. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status>
- [23] "Pascal Case vs. Camel Case Explained | Built In," *Built In*, 2024. <https://builtin.com/articles/pascal-case-vs-camel-case>
- [24] "Use JSDoc: Getting Started with JSDoc 3," *jsdoc.app*. <https://jsdoc.app/about-getting-started>
- [25] OWASP, "OWASP Top Ten," *owasp.org*, Jul. 2025. <https://owasp.org/www-project-top-ten/>
- [26] G. Lindemulder and M. Kosinski, "What is role-based access control (RBAC)?," *IBM*, Aug. 20, 2024. <https://www.ibm.com/think/topics/rbac>
- [27] Atlassian, "Gitflow Workflow," *Atlassian*, 2019. <https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow>
- [28] J. Lessard, "10 Essential Software Engineering Best Practices," *Axify.io*, Mar. 12, 2025. <https://axify.io/blog/software-engineering-best-practices>
- [29] M. Rehkopf, "Scrum Sprints," *Atlassian*, 2025. <https://www.atlassian.com/agile/scrum/sprints>